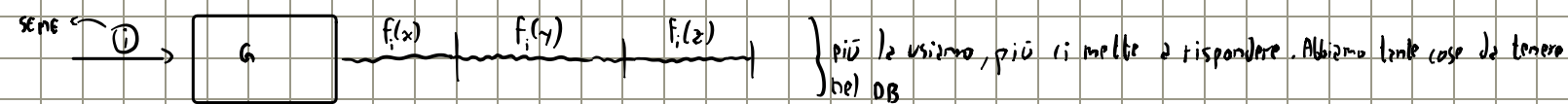


$$f_k \subseteq U_k$$

- 1) $|f_k| = 2^k$
- 2) $\exists$ algo per $(i, x) \mapsto f_i(x)$

$$3) \forall p \text{ per } |p_k^{0,0} - p_k^{0,1}| < k^{-w(i)}$$

↳ IDEA PER GENERARE UNA FUNZIONE CASUALE

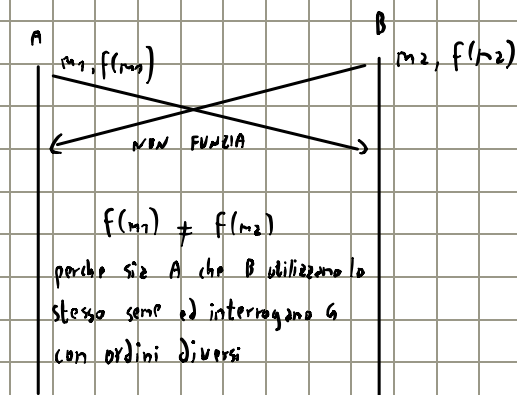


Su input x generiamo k bit e li restituiamo.
 Se successivo input y ,
 se $y = x$ restituiamo il risultato precedente
 Altrimenti generiamo altri k bit e li restituiamo

f_k che insieme è! la funzione dipende dall'ordine di chiamata, non dal suo indice i . Insomma dipende anche dagli argomenti (dagli input)
 Non rispetta la definizione di prima.

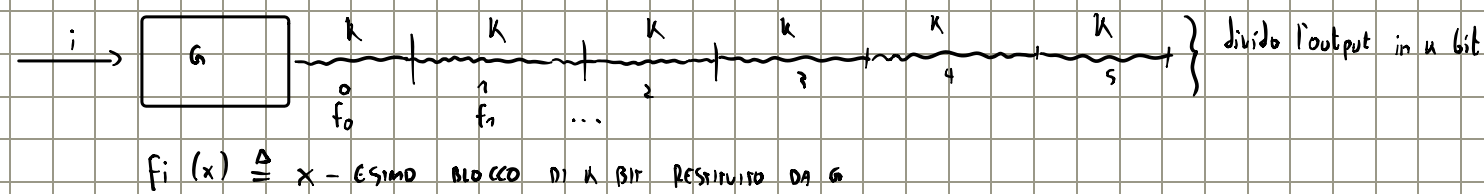
Inoltre non funziona per l'autenticazione di messaggi

$m, f_i(m)$



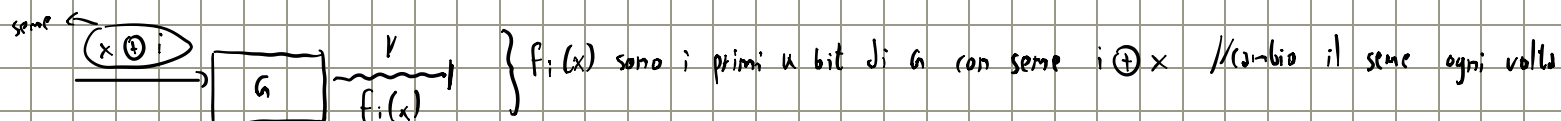
Utilizzare l'indice come oggetto segreto per distribuire a più agenti che hanno la copia locale di G non va bene con questo sistema

↳ PROVIAMO CON UN'IDEA DIVERSA



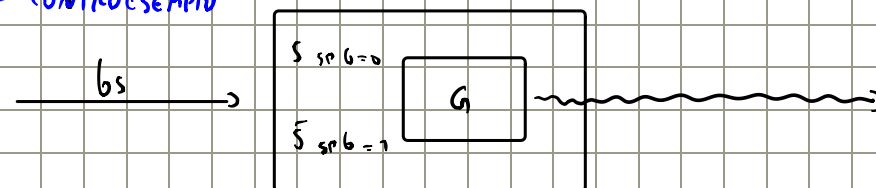
- rispetta il primo punto, le f sono definite in maniera univoca
- per il secondo punto abbiamo 2^k blocchi $f_i(1^k)$, il numero di bit da generare è esponenziale (in generale)
 Se vogliamo ottenere per esempio il 5 blocco

↳ LA FUNZIONE NON DEVE GENERARE TROPPI BIT

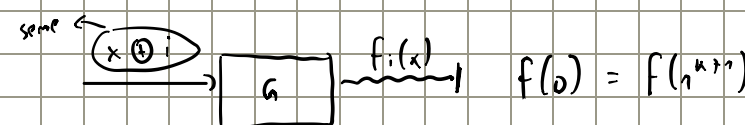


- rispetta il 1 punto
- " 2 punto
- non esiste il DISTINGUISHER?

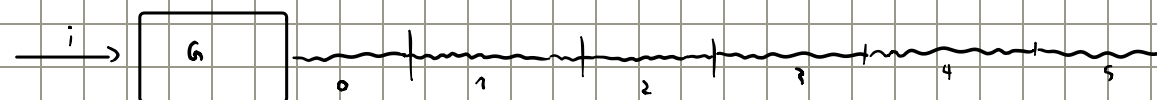
↳ CONTROESEMPIO



usiamo nel generatore precedente



↳ RIPRENDIAMO L'IDEA PRECEDENTE, AGGIUNGIAMO UN PEZZO



$f_i(x) = x$ -esimo gruppo

• primo bit in posizione kx

$$i(2^{kx+1}) \rightarrow \text{seme da usare per generare il primo bit del gruppo}$$

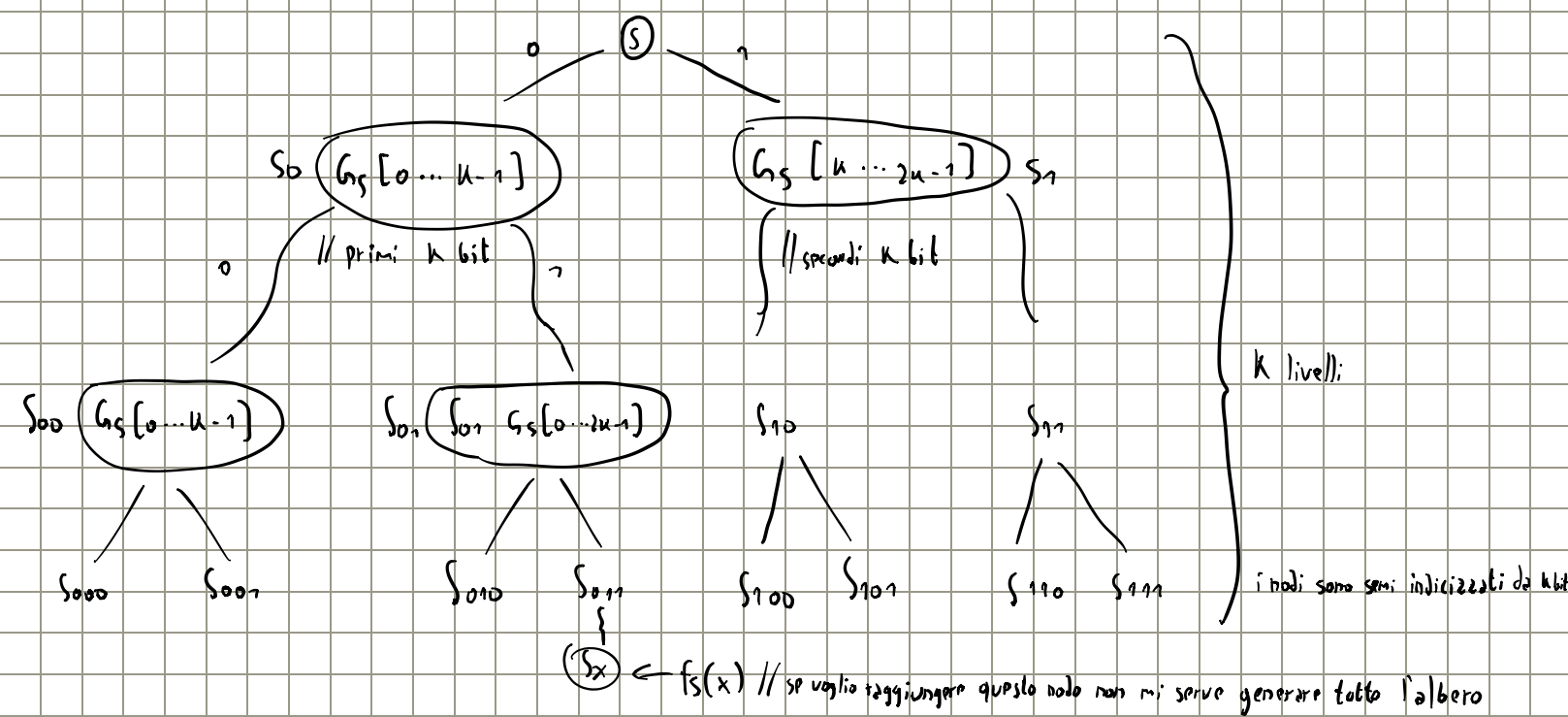
$$i(2^{kx+1}) \bmod \varphi(n) \bmod n$$

$i^2 \dots i^4 \dots i^8 \dots$

- ora in tempo polinomiale riusciamo a trovare
- ora proprietà 2 vale

↳

• Con il mio G faccio dei PES sul livello $i+1$
 in un caso si lavora con bit casuali
 nell'altro caso lavora con un seme casuale che genera i bit per arrivare a x

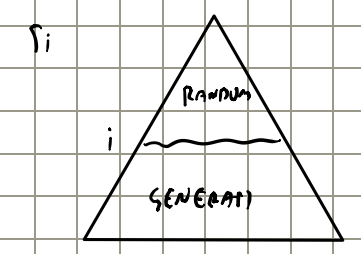


- per k volte genera $2k$ bit ($k \cdot 2k$)
- con RAS (elevamento al quadrato), $k \cdot 2k \cdot k^2$ il costo è in $k^4 \rightarrow$ alto per esisto
- ora il punto 3? $\Theta(k^4)$

↳ dim PER ASSURDO PER IL PUNTO 3

Per ogni livello $i = 0 \dots n$ definiamo l'albero τ_i e t_n .

- i nodi dei livelli $0 \dots i$ sono tutti casuali
- i nodi dei livelli $i+1 \dots n$ sono tutti generati dai livelli precedenti



τ_0 τ_k
 τ_k U_k
 $P_i = P_k^{Orr}$ // probabilità che 0 restituisca 1
 quando interagisce con una funzione
 generata secondo lo schema τ_i

sappiamo $|P_0 - P_k| > k^{-c}$

• ora con interpolazione sappiamo

$|P_i - P_{i+1}| > k^{-c'}$

↳ come sono fatti i due alberi

